ASSIGNMENT 4 - CO4

TASK 5 - DISTRIBUTED AND CLOUD-BASED STORAGE

7 Marks | Bloom's Level: K5 - Evaluate

1. Problem Context

The question describes a university that wants all student and staff files to be available from any campus and from home. The design must support shared-file access, acceptable read/write performance across networks, availability during component failures, appropriate consistency for concurrent users and manageable cost. The task asks for a comparison of NFS, distributed storage such as HDFS or Ceph, and cloud object storage such as Amazon S3 or Google Cloud Storage.

The question does not provide logical data volume, user count, request rate, network bandwidth, cloud region or provider quotations. Exact monthly cost and real response times therefore cannot be calculated from the source. This report uses equations and clearly labelled illustrations instead of invented measurements.

2. Task 5.1 - Compare the Options

Criterion	NFS	Distributed file system	Cloud object storage
Access model	Network-mounted file system with familiar paths, directories and permissions.	CephFS is POSIX-compliant; HDFS exposes a distributed API mainly for large-file processing.	Objects accessed by API/key and metadata; not inherently a POSIX shared folder.
Consistency	NFSv4 clients cache data/attributes, revalidate at OPEN and flush modified data at CLOSE; delegations coordinate some sharing.	CephFS coordinates metadata/client cache state through MDS capabilities. HDFS uses single-writer/write-pipeline rules and block replication.	Current S3 and Google Cloud Storage docs describe strong read-after-write and object-listing consistency.
Availability	Central server is a dependency. HA pairs, redundant storage and failover are needed to avoid an outage.	Replicas can serve reads after a node failure, subject to healthy metadata service, quorum and placement.	Provider-operated infrastructure is highly available, but users still depend on network, service and correct identity setup.
Performance	Good on a sized LAN; central CPU, network and disk can saturate as users grow.	Scales capacity/throughput across nodes. CephFS suits general file sharing; HDFS suits large sequential analytics.	Scales object workloads; remote latency depends on client-to-region path, API/auth and caching.
Cost	Server/storage/network/backup hardware, operations and staff.	More nodes/network/monitoring; replicas increase raw capacity and administration.	Usage charges for storage, requests, retrieval and data transfer; less local hardware to operate.

NFS is convenient when users expect normal file paths and shared folders. NFSv4 specifies cache validation, write flush on close and delegation behaviour; applications still need locking and concurrency rules for simultaneous updates.

CephFS is a POSIX-compliant distributed file system built on Ceph's object store, with metadata served by Metadata Servers and file data accessed through RADOS. It is a stronger fit than HDFS for general university home directories and shared documents. HDFS stores files as blocks across DataNodes, uses configurable replication and a write pipeline, and is primarily designed for large files and high-throughput processing.

Object stores are accessed through APIs rather than assuming each user can mount an ordinary shared directory. A university can provide a portal, sync client or application layer. Current S3 and Google Cloud Storage documentation describes strong read-after-write and listing consistency for object operations; cached public objects or access-control propagation can have separate timing considerations.

3. Task 5.2 - Replication and Consistency

Replication stores multiple copies on separate nodes or failure domains. With three replicas, one node failure does not remove the only copy: the system can continue reading from healthy copies and rebuild the missing replica. HDFS supports a configurable replication factor; its common factor of three uses rack-aware placement so data is not confined to one machine or rack. The write pipeline increases network traffic and disk use, but reads can be served from a nearby replica.

Property	Strong consistency	Eventual consistency
Meaning	After a successful write/commit, subsequent reads observe the latest committed value under the service contract.	Replicas may temporarily return older values; if updates stop and communication resumes, copies converge.
Benefit	Simpler collaboration and read-after-write behaviour.	Can reduce coordination and permit some operations during replica lag or network disruption.
Trade-off	Coordination/acknowledgement may add latency and can limit writes during partitions.	Applications may see stale values and need version checks, retries or conflict resolution.

Consistency is system-specific; replication does not automatically mean eventual consistency. CephFS coordinates metadata/cache state through MDS and client capabilities. HDFS uses a single-writer model and replica pipeline. Current S3 and Google Cloud Storage object operations offer strong read-after-write/listing consistency. Applications must still handle concurrent updates and multi-object workflows explicitly.

3.1 Capacity effect of three replicas

Let D be logical data volume. One copy requires about D raw capacity; three full replicas require about 3D, an extra 2D (200% overhead over the single-copy baseline). For an illustrative 10 TB dataset, that is 10 TB versus 30 TB raw capacity. These figures exclude metadata, reserve, snapshots and independent backups.

Replication is not a backup against accidental deletion, malware or a bad update copied to every replica. Keep versioned independent backups and test restoration.

4. Task 5.3 - OS Device Management for Storage

Component	Function	Importance
Device drivers	Translate generic OS requests into hardware commands; initialise devices and handle completions/errors.	Separates file systems from device-specific protocols and controllers.
Buffering	Temporarily holds data across speed differences, coalesces small writes and handles transfer alignment.	Smooths bursts and avoids inefficient small transfers.
Caching	Keeps frequently used file data/metadata in RAM; dirty write-back data is written later.	Reduces repeated I/O; dirty data must be flushed safely.
Spooling / queueing	Spooling queues work for a device/service. Storage stacks also queue asynchronous reads/writes in the block layer; classic spooling is most associated with printers.	Decouples producers from device timing and schedules queued work.
Block-device interface	Exposes storage as addressable logical blocks; file systems map file offsets to blocks and submit block I/O.	Common interface for HDD, SSD, NVMe and virtual block devices.

Linux's multi-queue block layer can stage requests in software queues and dispatch them to hardware queues via the driver. The file system handles names, directories, permissions and file-offset mapping; the block layer handles logical block I/O. Volatile device write caches require correct flush/FUA handling: a device can signal completion before data reaches non-volatile media, so the OS must issue required cache-flush or FUA operations for durability.

A read typically follows: application -> VFS/file-system lookup -> page-cache lookup -> block mapping on a cache miss -> block request queue -> driver -> device -> data returned to cache/application. A write may first update buffered/page-cache pages and be written back asynchronously; fsync/fdatasync requests the required persistence behaviour from the file system and device stack.

5. Alternative Concept - One Large NFS Server

A single NFS server is simpler and stores one primary copy, but all clients share its CPU, memory, network links, metadata service and disk subsystem. As the user count grows, the aggregate request rate approaches the server's

sustainable service rate; queue delay rises sharply near saturation. The server is also a single point of failure unless high availability, redundant storage and tested failover are implemented.

5.1 Read/write response as users grow

Let U be active users, lambda0 the average I/O request rate per user, and mu the sustainable service rate of the NFS server. Aggregate arrival rate is lambda = U x lambda0 and utilisation is rho = lambda / mu. For a simplified M/M/1 queue:

Mean response time W = 1 / (mu - lambda) = S / (1 - rho), where S = 1/mu is the idle mean service time. This is explanatory, not a measured NFS benchmark.

Server utilisation	Approximate mean response
50%	2 x idle service time
80%	5 x idle service time
95%	20 x idle service time

As users increase, cache hits can reduce disk I/O, but misses still reach the central service. Writes require server processing, storage bandwidth and durability operations. Once aggregate demand reaches or exceeds mu, requests accumulate or must be throttled/rejected. A balanced distributed cluster can increase aggregate capacity by spreading data across nodes, but metadata, network, hot files, skew and replica acknowledgement may become bottlenecks.

5.2 One copy versus three replicas

Architecture	Raw capacity for data D	Implication
Single NFS server / one copy	D	Smallest primary-copy requirement; independent backup/HA adds capacity.
Distributed / three replicas	3D	Two extra copies; about 200% raw overhead; placement across failure domains.

For example, a 10 TB logical dataset uses about 10 TB on a single primary copy and 30 TB for three full replicas before metadata and reserve. The 10 TB number is illustrative; the task supplies no actual data volume.

5.3 Availability when one server fails

If the only NFS server fails, clients cannot access the namespace until recovery or failover. If the storage subsystem fails, data can be unavailable or lost without independent redundancy. A replicated distributed system can keep serving from healthy copies after a storage-node failure, provided enough replicas and healthy metadata/quorum remain. It may operate in degraded mode while rebuilding the missing copy.

5.4 Pseudocode for the alternative

```
SINGLE NFS SERVER:
  receive request; authenticate and check permissions
  enqueue on the single server's I/O service
  READ: check cache; on miss read storage; return data
  WRITE: buffer/commit by stable-write policy; acknowledge
  server failure: service stops until recovery/failover

THREE-REPLICA DISTRIBUTED:
  WRITE(file, data):
      locate metadata; choose 3 healthy nodes
      pipeline/parallelise data to replica set
      wait for configured durability acknowledgements
      commit metadata; acknowledge client
  READ(file, offset):
      locate healthy replicas; choose a nearby copy
      read and verify; return data
  NODE FAILURE:
      continue from healthy replicas if policy/quorum permits
      detect missing copy; rebuild on a healthy node
```

5.5 Complexity comparison

Operation	Single NFS	Three-replica distributed
Per-request routing	Approximately O(1) average with pre-indexed namespace; queue delay grows with utilisation.	Metadata/placement lookup plus replica selection; indexed namespace lookup may be O(log N).
Data movement	O(B) primary data write for B bytes.	Approximately O(r x B) replica-network work for replication factor r=3.
Read/failure	One server/cache path; one server failure stops service without HA.	Choose a healthy/nearby replica; one node failure can be tolerated if replica and metadata health remain.

Big-O describes lookup/data movement, not wall-clock latency. The single-server queue model exposes the scaling risk: W = 1/(mu-lambda) rises as lambda approaches mu. Distributed storage pays extra capacity (3D) and network work to gain fault tolerance and aggregate read capacity.

6. Cost Evaluation and Recommendation

The assignment does not supply data volume, concurrency, request ratio, campus bandwidth, cloud region or quotations. An accurate monthly currency cost cannot be calculated without inputs. Compare:

```
NFS monthly cost:
  server hardware + primary storage(D) + network
  + HA/backup storage + power + administration

Three-replica cost:
  storage nodes + raw storage(3D) + network
  + metadata/monitoring + backup + administration

Cloud object cost:
  stored GB-month + object requests + retrieval (if applicable)
  + network egress + identity/gateway/backup/management
```

Cloud object storage reduces local hardware operations, but billing depends on GB-month, request volume, home-user downloads, region and storage class. Egress can matter when files are downloaded repeatedly. Use the provider price calculator with measured usage.

Recommended design: use a distributed POSIX file system such as CephFS for the university's primary shared-file workspace when conventional directories, permissions and file operations are required across campuses. Place replicas across independent nodes and, where possible, racks; maintain highly available metadata services; integrate identity, quotas and audit logs; and retain independent tested backups. HDFS is better suited to large sequential analytics files than interactive home directories. Cloud object storage is a useful complementary layer for archives or unstructured data behind a portal/sync application.

If NFS is selected for simplicity and the workload is modest, deploy an HA pair with redundant storage/network paths, client caching and independent backups. One unprotected NFS server should not be the only copy of university-wide files.

7. Conclusion

NFS offers familiar shared-file access but centralises performance and failure risk unless HA is added. Distributed storage spreads data and workload across nodes; three replicas improve node-failure tolerance but require about three times raw capacity. CephFS fits general POSIX-style university files, while HDFS is oriented to large-file throughput. S3 and Google Cloud Storage offer strong object read-after-write consistency, but a portal/sync layer is needed for ordinary file sharing and request/egress costs must be assessed. OS drivers, buffers, caches, queues and the block interface coordinate device I/O beneath the file system.

8. Technical References

[1] RFC 7530, *Network File System (NFS) Version 4 Protocol*, client caching, data revalidation, close/flush rules and delegations. https://www.rfc-editor.org/rfc/rfc7530.html

[2] Ceph Documentation, *Ceph File System*, POSIX compliance, RADOS and metadata servers. https://docs.ceph.com/en/latest/cephfs/

[3] Ceph Documentation, *Distributed Metadata Cache*, capabilities and client cache coordination. https://docs.ceph.com/en/latest/cephfs/mdcache/

[4] Apache Hadoop, *HDFS Architecture*, block replication, rack awareness and write pipeline. https://hadoop.apache.org/docs/r3.2.2/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[5] Amazon S3, *Strong Consistency*, read-after-write and LIST consistency. https://aws.amazon.com/s3/consistency/

[6] Google Cloud, *Cloud Storage Consistency*, strong consistency and cache qualifications. https://docs.cloud.google.com/storage/docs/consistency

[7] Linux Kernel Documentation, *Multi-Queue Block IO Queueing Mechanism*. https://docs.kernel.org/block/blk-mq.html

[8] Linux Kernel Documentation, *Explicit Volatile Write Back Cache Control*. https://docs.kernel.org/block/writeback_cache_control.html

Source alignment: Assignment 4 - CO4, Task 5 - Distributed and Cloud-Based Storage, 7 Marks, page 39. General task instructions appear on pages 40-41.